

IBENTO: A Scalable Centralized Campus Event Management System

Ahlada A Adrija
Dept. of Computer Engineering
College of Engineering Chengannur
Alappuzha, India
CHN23CS021

Aswinraj
Dept. of Computer Engineering
College of Engineering Chengannur
Alappuzha, India
CHN23CS053

Govind J S
Dept. of Computer Engineering
College of Engineering Chengannur
Alappuzha, India
CHN23CS089

Sourav S
Dept. of Computer Engineering
College of Engineering Chengannur
Alappuzha, India
CHN23CS180

Abstract—Traditional university campus event management relies heavily on fragmented, manual, and paper-dependent workflows, resulting in coordination delays, operational data silos, and heightened security vulnerabilities such as unauthorized event modifications. This paper introduces IBENTO, a unified, cloud-native web platform designed to streamline and automate the complete campus event lifecycle. Built on a high-concurrency architecture using a React.js single-page frontend and a serverless Firebase backend, the system implements Custom Claims-based Role-Based Access Control (RBAC) to enforce strict data isolation among Students, Club Leads, and SuperAdmins. To mitigate critical race conditions during peak registration hours, an Optimistic Concurrency Control mechanism utilizing atomic database transactions was developed. Furthermore, the platform integrates real-time state synchronization via the Observer pattern and secure, temporary document routing through localized Blob URL generation. Our deployment confirms a highly resilient, paperless ecosystem that significantly reduces institutional material waste while enhancing co-curricular administrative throughput.

Index Terms—Cloud-Native Architecture, Role-Based Access Control, Real-Time Synchronization, Optimistic Concurrency Control, Event Management Systems.

I. INTRODUCTION

Educational institutions frequently organize an array of co-curricular workshops, seminars, cultural programs, and technical fests across diverse student forums and departments[cite: 11]. However, administering these activities through scattered channels or physical documentation routinely causes severe operational inefficiencies, scheduling conflicts, and communication failure[cite: 12, 14]. Existing commercial event tools fail to address college-level administrative governance, often lacking multi-tiered authorization hierarchies and verification frameworks tailored for institutional compliance[cite: 13, 19].

To bridge this structural gap, this paper presents IBENTO, a customized, smart campus event management system[cite: 2, 15]. The system consolidates user authentication, dynamic data presentation, payment tracking, and digital check-ins

into an interconnected cloud platform[cite: 56, 137, 140, 141]. The core focus of this research centers on implementing advanced software engineering patterns—such as token-embedded access controls, web socket observers, and transactional race-condition mitigation—to provide an optimized, paperless ecosystem for academic communities[cite: 89, 92, 101, 135].

II. SYSTEM ARCHITECTURE

The architectural blueprint of IBENTO is structured as a decoupled, four-tier, cloud-native framework engineered to minimize client-side computational latency, ensure strict data isolation, and handle heavy transactional workloads without operational degradation. The structural decoupling ensures that presentation-tier modifications do not compromise backend state machines or introduce data validation gaps. The holistic system boundaries, actor responsibilities, execution authorities, and operational flows across these layers are mapped structurally through the system's core functional specifications in Fig. 1.

A. Multi-Tier Architectural Breakdown

The system divides responsibilities across four distinct layers to promote maintainability, ease debugging vectors, and horizontally scale independent cloud services:

- **Presentation Layer (Client-Side Interface):** Comprises isolated, role-guarded user interfaces tailored for Students, Club Leads, and SuperAdmins. This tier relies on component-scoped state management to handle reactive rendering based on user interaction, isolating administrative panels from standard client views.
- **Frontend Application Core (React.js Engine):** Built as a Single Page Application (SPA) leveraging a Virtual DOM component framework. Rather than performing resource-intensive direct manipulations on the browser's

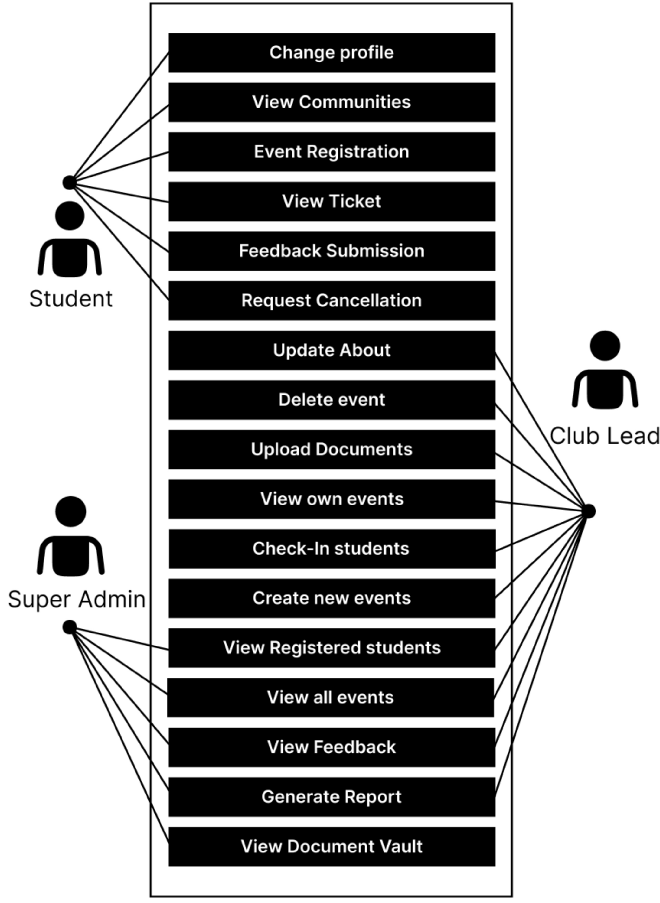


Fig. 1. IBENTO System Core Use Case Specification.

real Document Object Model (DOM), mutations are updated on a lightweight memory tree. A reconciliation algorithm computes minimal diff matrices, updating the real DOM via batched operations to prevent layout reflow bottlenecks during fast data streams.

- **Serverless Business Logic Layer:** Executed via event-driven, stateless Firebase Cloud Functions running on isolated V8 engine containers. This framework eliminates the operational overhead of traditional monolithic servers, dynamically spinning up execution instances on demand. Media tracking, high-overhead computations, and report generations are completely offloaded to this tier.
- **Data Infrastructure Tier:** Utilizes a hybrid storage design. Structured application states and relational mappings are handled via a denormalized, NoSQL Firestore document database. Unstructured binary assets (e.g., event graphics, receipts, documents) are offloaded to Cloudinary endpoints for automatic on-the-fly edge compression, formatting optimizations, and delivery via global Content Delivery Networks (CDNs).

B. NoSQL Data Modeling and Collections Schema

To guarantee ultra-low latency reads during traffic spikes, the database schema deliberately avoids deeply nested relational tables and heavy runtime JOIN queries in favor of a denormalized collection-document pattern. Data entities are separated into three core structural layers:

- **Users Collection:** Root-level documents keyed by unique authentication identifiers, storing validated institutional email structures, profile attributes, and indexed arrays mapping to registered events.
- **Events Collection:** Parent documents containing state vectors such as title, organization identifiers, strict capacity limits, cost parameters, and seat inventories.
- **Registrations Sub-collection:** Scaled hierarchically directly beneath individual event document paths. Nesting these documents isolates high-frequency concurrent writes to distinct collection fragments, avoiding lock contention across the rest of the database environment.

III. METHODOLOGY & TECHNICAL IMPLEMENTATION

The execution stability and operational correctness of IBENTO rely on a strict convergence of distributed systems paradigms, cryptographic identity evaluations, and transactional boundary enforcement. The full runtime execution sequence, cross-tier communication links, and state lifecycle transitions of a user registration flow across these decoupled components are modeled sequentially in Fig. 2.

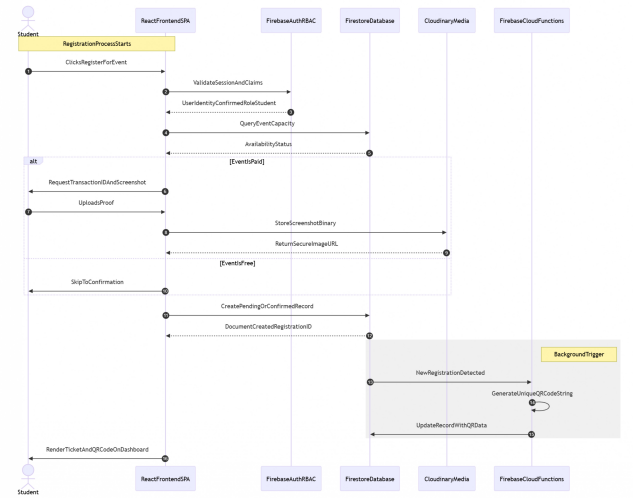


Fig. 2. Sequence Diagram of the Event Registration and Verification Flow.

A. Custom Claims-Based Authorization (RBAC)

Traditional monolithic architectures rely on stateful sessions or repetitive middleware database queries to verify user roles before executing restricted actions. To eliminate this API overhead, IBENTO implements an asynchronous, cryptographic role validation model directly inside token signatures.

Upon user signup, a secure cloud-side authentication trigger validates the user's domain email. If verified as an administrative entity, a server-side script updates the identity provider

metadata to embed custom JSON-based boolean claims attributes directly within the user’s JSON Web Token (JWT) signature. The claims structure acts as an immutable payload containing boolean permissions variables for student, club leader, and super-administrative roles.

When a client process requests a database resource, the client library passes this signed JWT within the header. The database layer decodes the token using public keys and evaluates the internal attributes against declarative server-side security rules natively. This design guarantees that authorization checks execute in $O(1)$ time complexity, ensuring absolute structural safety even if the client-side code is compromised or modified.

B. Real-Time State Synchronization Engine

To keep the presentation layer unified across thousands of concurrent active devices without resorting to performance-heavy client-side HTTP short-polling, the synchronization architecture relies on the Observer design pattern.

The React client library initiates long-lived TCP connection channels via persistent WebSockets directly to the document repository. When a component registers a data listener using specialized snapshot hooks, the server pushes the initial query state and caches the transaction token locally. The moment any cloud document is mutated by a concurrent writer, the database engine computes a micro-delta containing only the modified fields. This small delta payload is instantly pushed through the open WebSocket, updating the localized client-side store and triggering a precise, partial component re-render without page reloads.

C. Conditional Skip-Logic Gateways

To handle variations between registration workflows for different types of campus activities, the registration system utilizes a strict conditional skip-logic pipeline. When a user requests a registration session for an event instance x , the application evaluates a deterministic boolean function:

$$P(x) = \begin{cases} 1 & \text{if } x.\text{isPaid} = \text{false} \\ 0 & \text{if } x.\text{isPaid} = \text{true} \end{cases}$$

If $P(x) = 1$, the application clears the transaction overhead, routes execution around the verification containers, and instantly appends the user record to the confirmed attendee array. If $P(x) = 0$, a state-machine interceptor suspends the registration context, locks the view to a payment documentation upload screen, and prevents further processing until binary transaction metadata is committed to the cloud storage bucket.

D. Optimistic Concurrency Control via Atomic Transactions

During high-volume institutional registration intervals (such as limited-capacity technical bootcamps), hundreds of parallel clients may read identical seat quantities simultaneously. If concurrent processes attempt to modify remaining seats without absolute locking, a race condition occurs, resulting in database overbooking anomalies.

To solve this distributed conflict, IBENTO rejects slow, resource-heavy pessimistic database locks and instead implements an Optimistic Concurrency Control (OCC) mechanism powered by atomic transactions. Transactions execute using an isolated Read-Verify-Write sequence. During the read phase, the transaction samples the target document’s capacity parameter. The verification phase then evaluates the logical boundary condition:

$$C_{\text{current}} - \Delta_{\text{requested}} \geq 0$$

If this condition fails, the transaction is instantly aborted, and an out-of-bounds capacity exception is returned to the client presentation layer. If verification succeeds, the write phase decrements the seating variable and appends the registration object into the sub-collection within a single atomic operation. If a parallel thread updates the specific event document between the read and write phases, the database catches a version-tag mismatch. The transaction immediately rolls back all partial writes to preserve database consistency, and automatically retries the loop with updated snapshots up to five times before throwing a contention exception.

E. Temporary Document Vault Masking

For internal administrative files, accounting logs, and institutional clearance certificates uploaded by club chairs, raw URL exposure risks file scraping and unauthorized access. IBENTO wraps these secure cloud storage elements using client-side object memory masking.

When an authorized user requests a file view, the cloud function evaluates the actor’s custom JWT claim. Upon validation, the cloud storage bucket streams the file down as a raw binary array buffer via HTTPS. The client engine handles this stream in memory and converts it into an isolated, local browser reference using an internal blob initialization routine:

`SecureBlobURL = URL.createObjectURL(binaryDataStream)`

The generated address exposes a virtual link prefixed with local blob identifiers that exists only within the local browser’s runtime tab thread. Because it points directly to an encrypted heap allocation rather than a public web link, the cloud infrastructure path remains completely hidden from inspect-element operations. The link expires completely the moment the tab session is closed or destroyed.

IV. SOCIETAL ALIGNMENT & SUSTAINABILITY

In accordance with institutional mapping guidelines, the development parameters of IBENTO directly advance three United Nations Sustainable Development Goals (SDGs)[cite: 124]:

- **SDG 9 (Industry, Innovation, & Infrastructure):** Modernizes campus administrative infrastructure by deploying high-concurrency cloud technologies and automated serverless logic routines over obsolete legacy procedures[cite: 134].
- **SDG 12 (Responsible Consumption & Production):** Completely eliminates systemic material waste within the

institution by converting physical event tokens, flyers, and paper feedback slips into carbon-neutral digital dashboards and automated QR credentials[cite: 135].

- **SDG 4 (Quality Education):** Streamlines co-curricular resource management, allowing student bootcamps and engineering forums to seamlessly execute skill-development and technical training seminars, thus widening access to experiential engineering education[cite: 136].

V. CONCLUSION AND FUTURE SCOPE

IBENTO successfully implements a resilient, cloud-native paradigm for institutional event optimization[cite: 146]. By anchoring frontend performance to modular components and backend workflows to atomic transactions and granular security models, the architecture demonstrates a massive leap over manual administrative systems[cite: 101, 112, 117, 147]. Future iterations will focus on integrating automated email notification protocols and native multi-tenant payment gateways to expand operational boundaries across wider university networks[cite: 63, 83].

REFERENCES

- [1] Design and Implementation of a Comprehensive College Event Management System, International Journal of Computer Applications (IJCA), 2025.
- [2] College Event Management System, International Research Journal of Engineering and Technology (IRJET), Vol. 7, No. 3.
- [3] Exploring Front End Framework React: A Review, IRJET, Vol. 7, No. 7.
- [4] Firebase in App Development, IRJET, Vol. 3, No. 12.